JAMAL MOHAMED COLLEGE (AUTONOMOUS)

Accredited with 'A++' Grade by NAAC (4th Cycle) with CGPA 3.69 out of 4.0
(Affiliated to Bharathidasan University)
TIRUCHIRAPPALLI – 620 020

DEPARTMENT OF COMPUTER SCIENCE & IT (SF)

B.Sc. INFORMATION TECHNOLOGY WITH CYBERSECURITY

SEMESTER – VI

CYBER DEFENSE LAB (23UIC6DE3BP)

CERTIFICATE

Name : _____

Register Number : _____

This is to certify that this is a bonafide record of practical work done in the Computer Centre of Jamal Mohamed College, Tiruchirappalli - 20 during the year 2025- 2026.

Staff In-charge

Submitted for the B.Sc. Information Technology with Cybersecurity Practical Examination held at Jamal Mohamed College, Tiruchirappalli - 20, on _____.

Tiruchirappalli –20

Date :

Internal Examiner **External Examiner**

CONTENTS

Sl. No.	Date	Title of the Program	Page No.	Remarks
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				

ENCRYPTION_UNWRAPPED

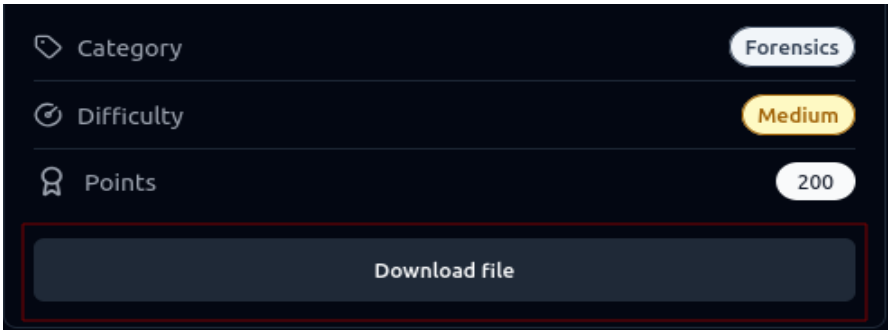
DESCRIPTION:

We found this packet capture and key. Recover the flag.

INTRODUCTION:

We have given a pcapng file we can open it with wireshark to analyse the network traffic to retrieve the flag

We can download the file using the download file button



We have given a zip file we can unzip it

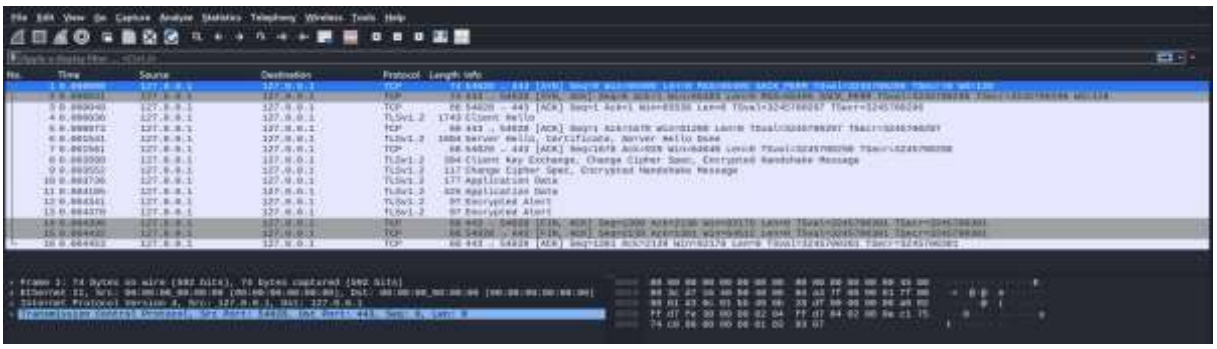
```
┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
Challenge_file.tar
```

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# tar -xf Challenge_file.tar

┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
Challenge_file  Challenge_file.tar
```

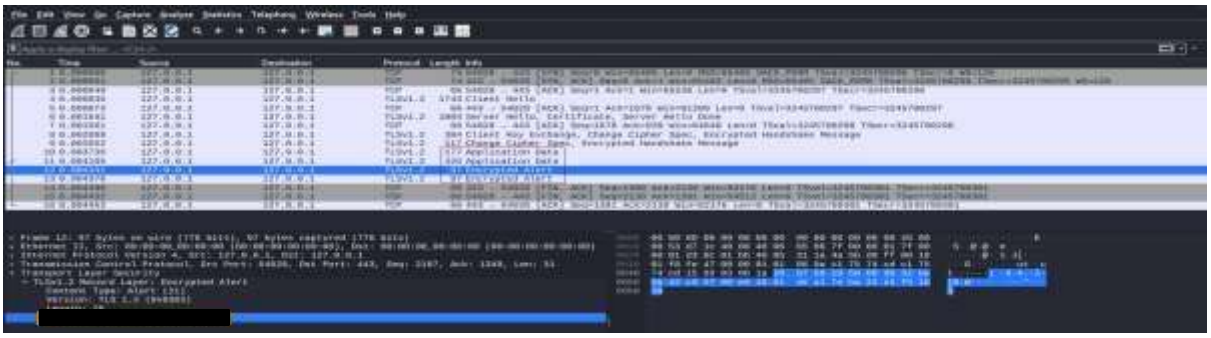
We have given two files one is a pcapng file and another one is a key file

```
┌──(root㉿kali)-[/home/kali/Documents/Challenge_file]
└─# ls
capture.pcap  server.key
```

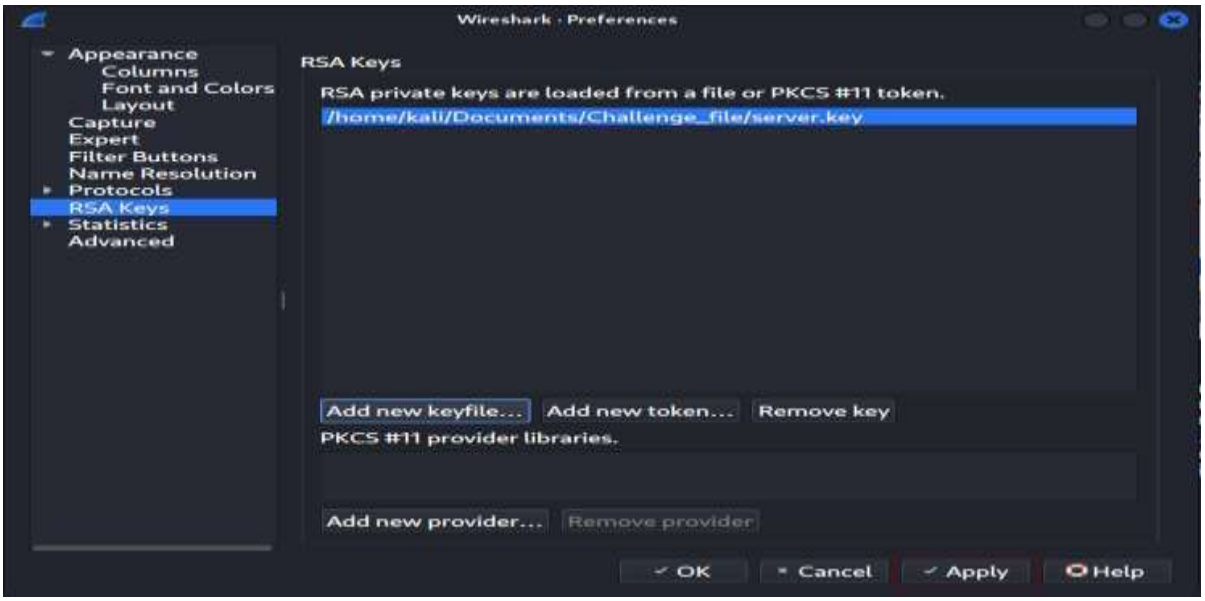
We can open the pcap file using wireshark and analyse the network traffic



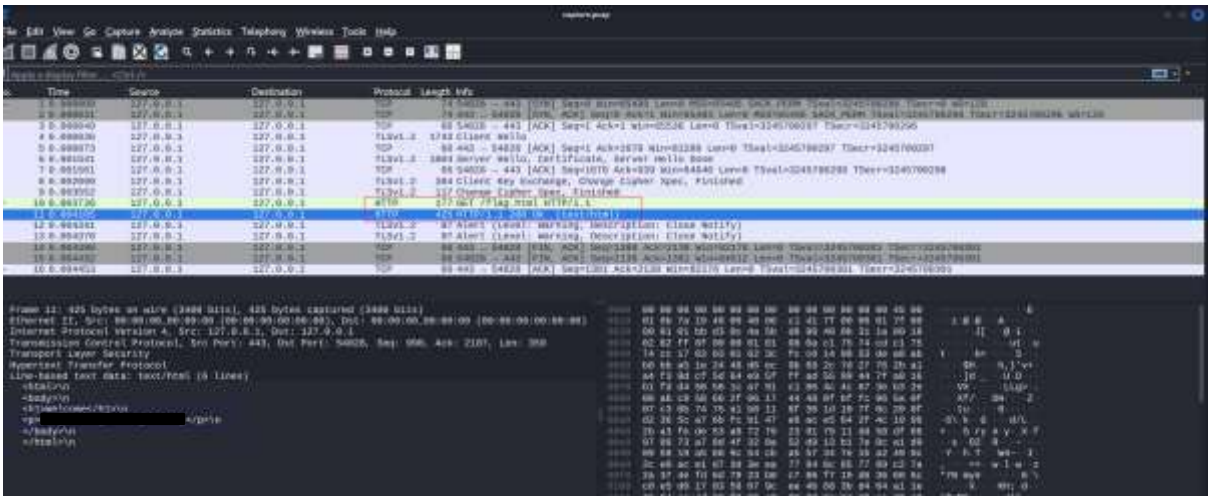
We can look at all the packets one by one as there is only 16 packets



It looks like the packets contains some application data but it is all encrypted. We can use the server.key to decrypt them using Edit → Preferences → RSA Keys



Apply the key and look at the traffic, if no changes has been happened close the wireshark and open again



We can see that we have HTTP packets now, which contains the flag

CT[redacted]X}

Butter_in_the_Wire

DESCRIPTION:

A chef is suspected to have leaked the recipe of a pastry company. Find the exfiltrated data and recover the recipe (flag).

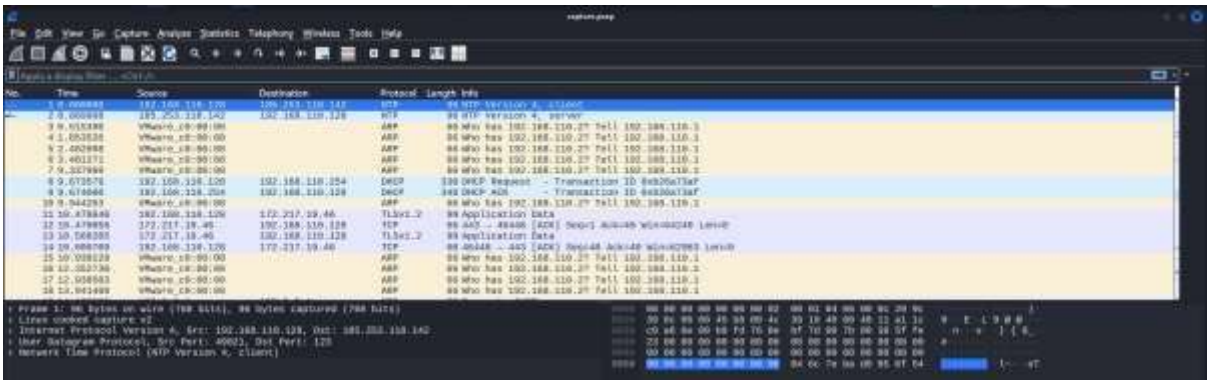
INTRODUCTION:

We can use wireshark tool to analyse the pcapng file to take a look at the network traffic and filter for data to exfiltrate the flag

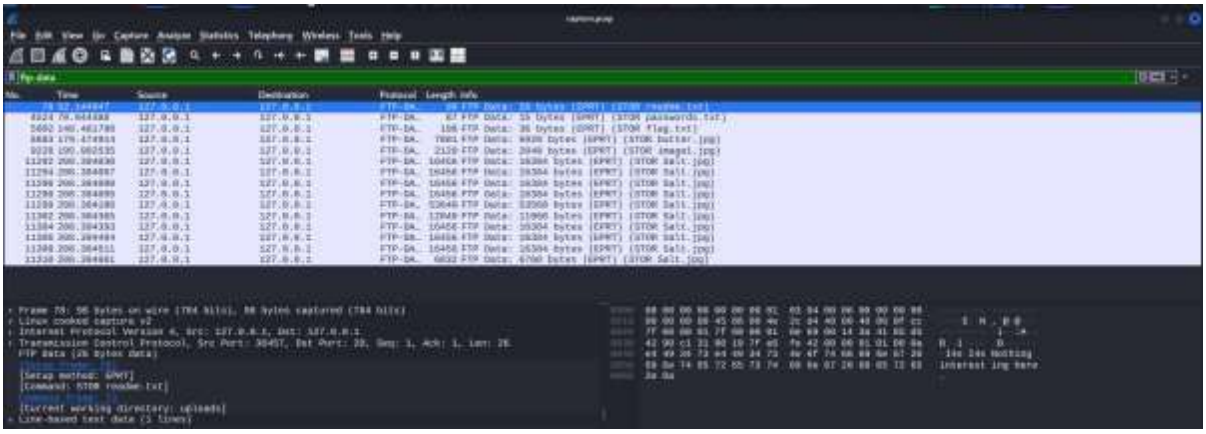
Download the file

We can see that it is a pcapng file so we can open it with wireshark

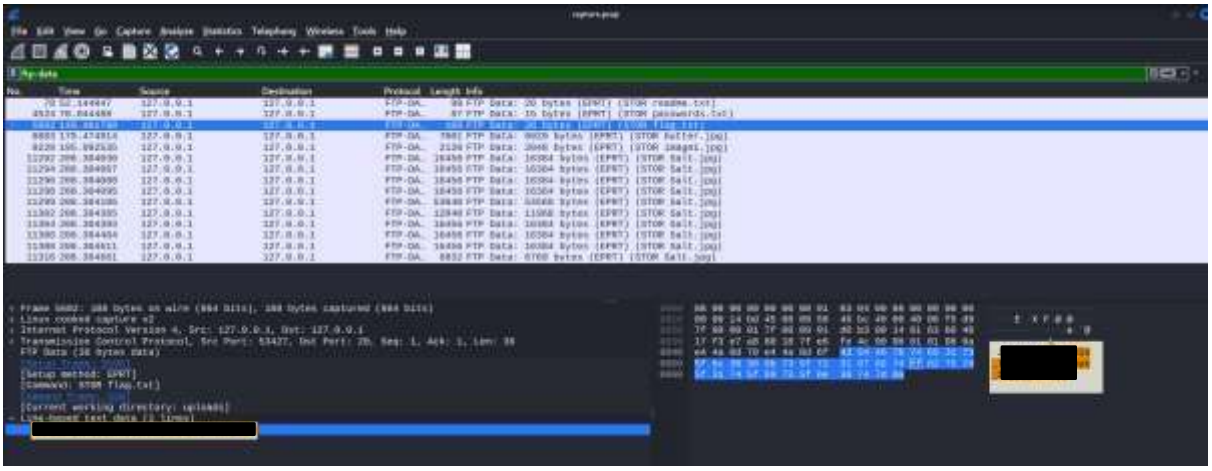
```
┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
capture.pcap
```



Now with the hint and description we can filter and search for ftp-data packets using the filter option



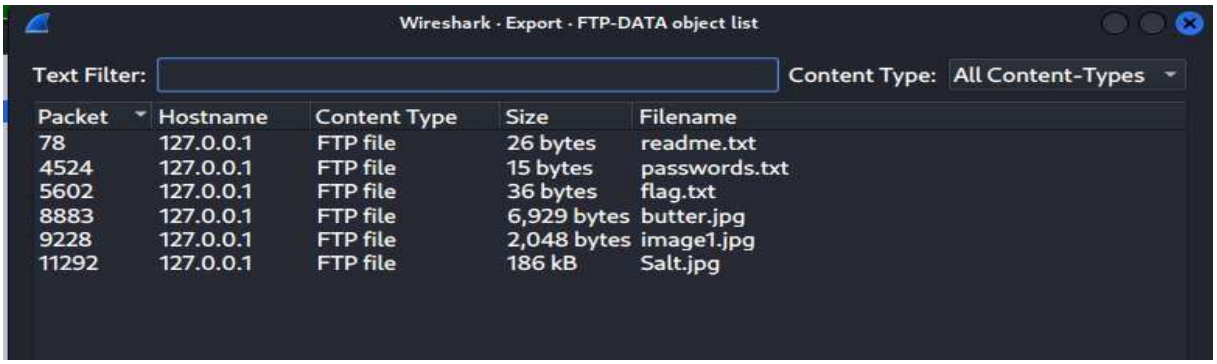
We can look at the data file one by one to find the flag



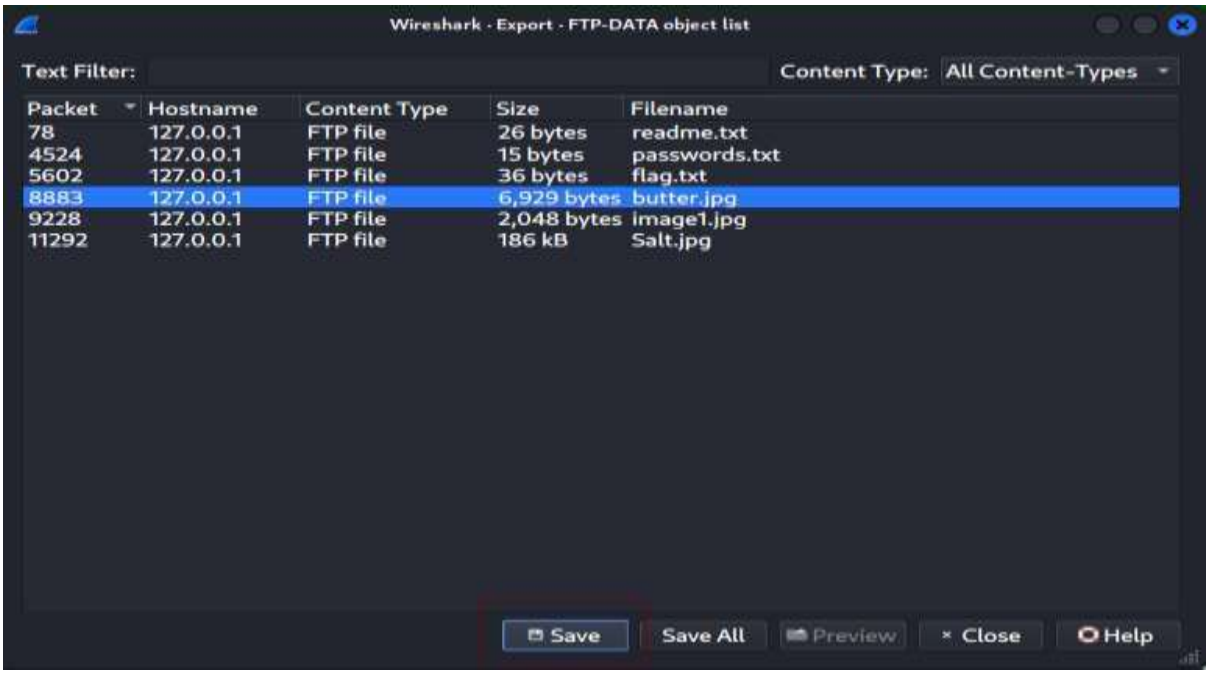
We can see that we have a fake flag, all the packets looks similar other than the packet 8883 butter.jpg, we can try to export the jpg file using the export option

Go to file --> Export-Objects --> FTP-DATA

We can select the butter.jpg file and export it

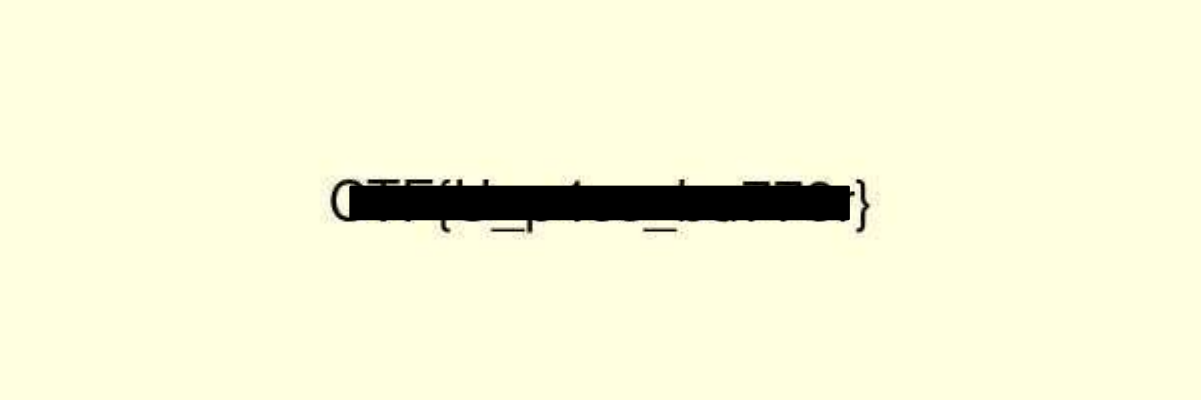


We can save the file in the directory we want, with the name we like

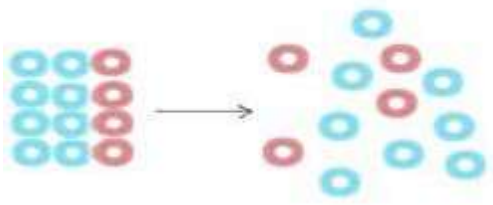


We can take a look at the save butter.jpg

Now we can open the file using the file system or in the terminal with command '**open <filename>**'



Submit the flag to finish the challenge



ENTROPY

DESCRIPTION:

A routine vulnerability scan completed successfully, yet something remains unresolved

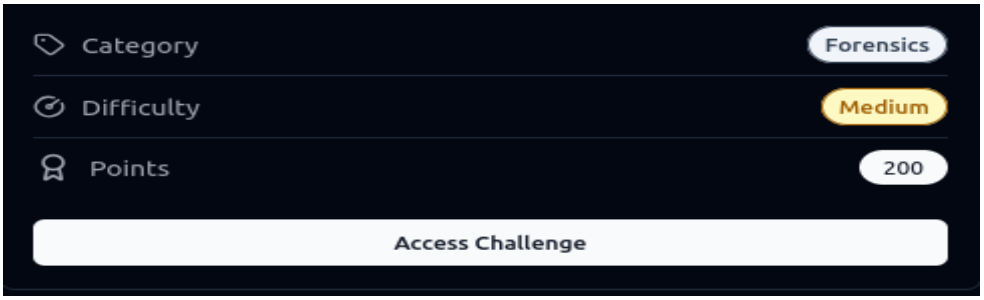
INTRODUCTION:

The flag is divided into three parts, where each of them can be found with the help of these three keywords: "Invisible", "Dead", "Knowledge".

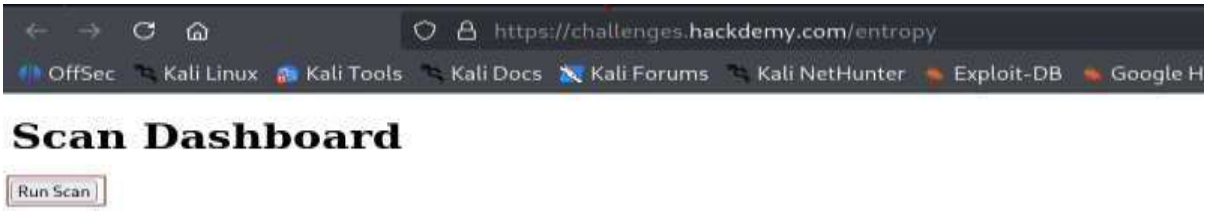
Unreachable assets may still generate metadata.

Knowledge base integrity varies by host.

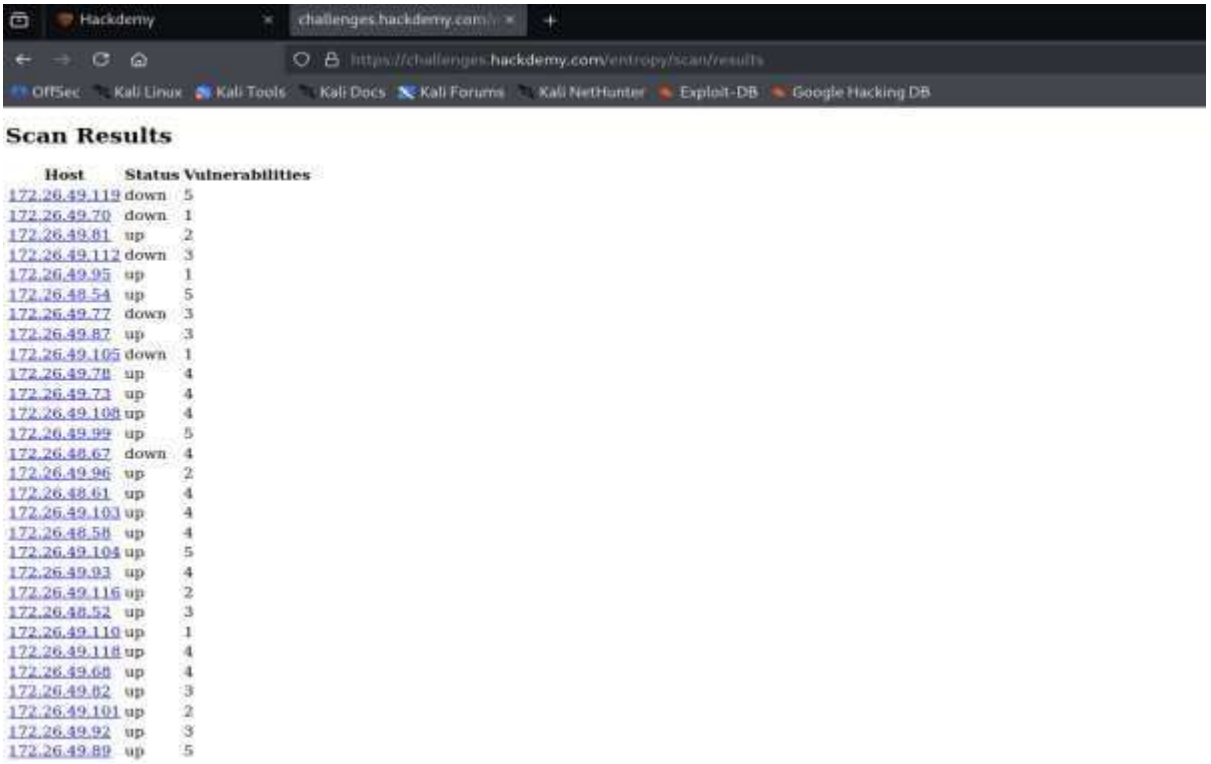
We can access the challenge using the access challenge button



Use the run scan button to start the scan



We can see we have a vulnerability scan result



Hackdemy | challenges.hackdemy.com

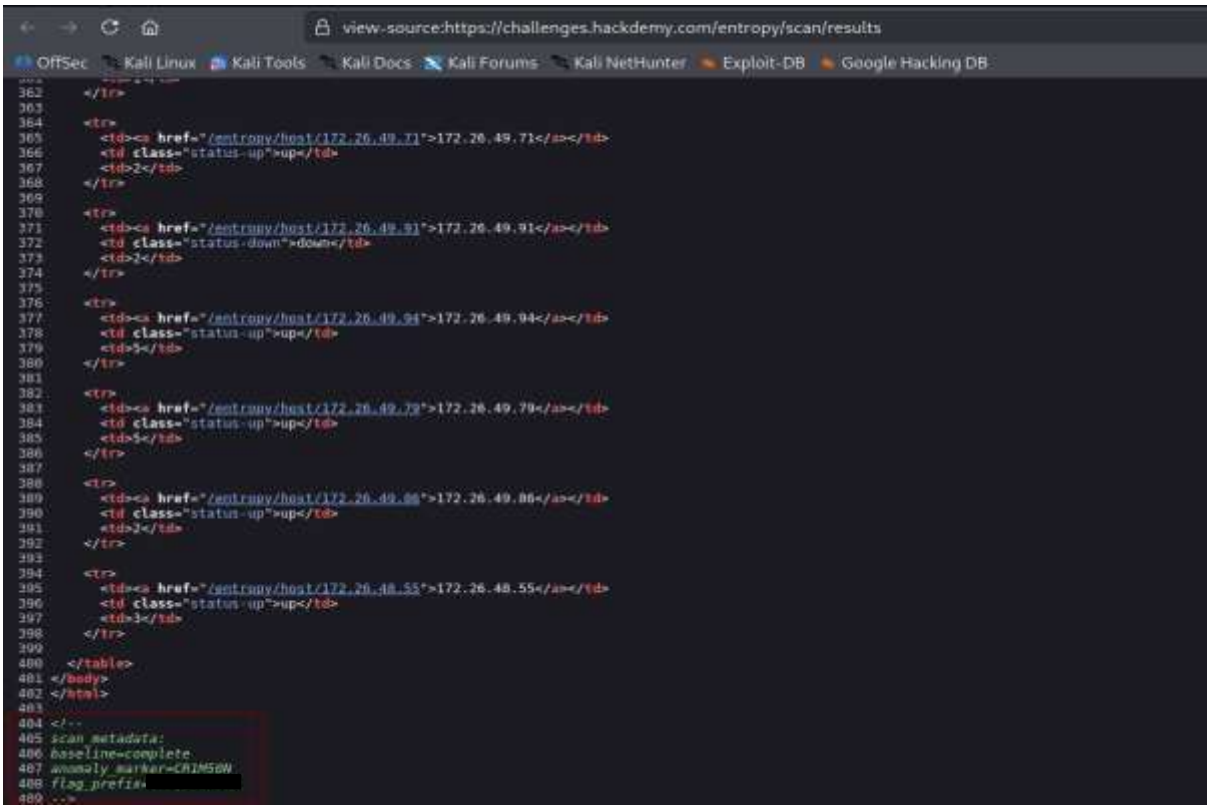
https://challenges.hackdemy.com/entropy/scan/results

OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

Scan Results

Host	Status	Vulnerabilities
172.26.49.119	down	5
172.26.49.70	down	1
172.26.49.81	up	2
172.26.49.112	down	3
172.26.49.95	up	1
172.26.48.54	up	5
172.26.49.77	down	3
172.26.49.87	up	3
172.26.49.105	down	1
172.26.49.78	up	4
172.26.49.73	up	4
172.26.49.108	up	4
172.26.49.99	up	5
172.26.48.67	down	4
172.26.49.96	up	2
172.26.48.61	up	4
172.26.49.103	up	4
172.26.48.58	up	4
172.26.49.104	up	5
172.26.49.93	up	4
172.26.49.116	up	2
172.26.48.52	up	3
172.26.49.110	up	1
172.26.49.118	up	4
172.26.49.68	up	4
172.26.49.82	up	3
172.26.49.101	up	2
172.26.49.92	up	3
172.26.49.89	up	5

The first flag hint is invisible we can try to take a look at source code for hint



view-source:https://challenges.hackdemy.com/entropy/scan/results

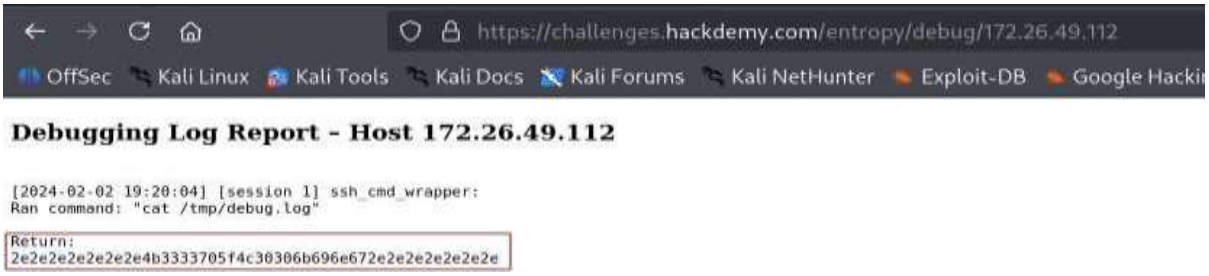
OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

```
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.49.71">172.26.49.71</a></td>
      <td class="status-up">up</td>
      <td>2</td>
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.49.91">172.26.49.91</a></td>
      <td class="status-down">down</td>
      <td>2</td>
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.49.94">172.26.49.94</a></td>
      <td class="status-up">up</td>
      <td>5</td>
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.49.79">172.26.49.79</a></td>
      <td class="status-up">up</td>
      <td>5</td>
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.49.86">172.26.49.86</a></td>
      <td class="status-up">up</td>
      <td>2</td>
    </tr>

    <tr>
      <td><a href="/entropy/host/172.26.48.55">172.26.48.55</a></td>
      <td class="status-up">up</td>
      <td>3</td>
    </tr>

  </table>
</body>
</html>

<!--
scan_metadata:
baseline=complete
anomaly_marker=CRIMSON
flag_prefix=[redacted]
-->
```

We can see that we have the first part of flag **invisible** which is hidden in source code is retrieved

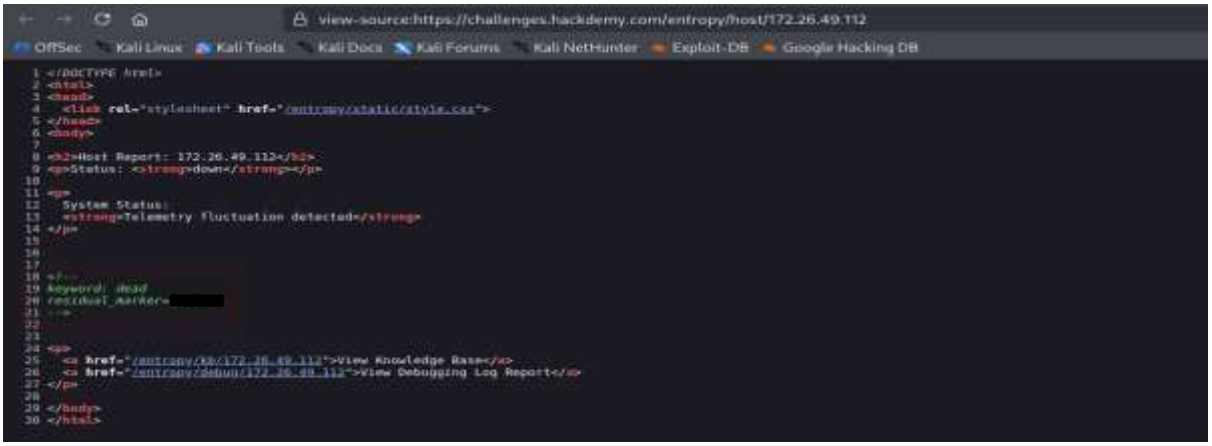
Now we can try to retrieve the second part dead, we can take a look at the hosts which are down

We can take a look at the knowledge base and debugging log report



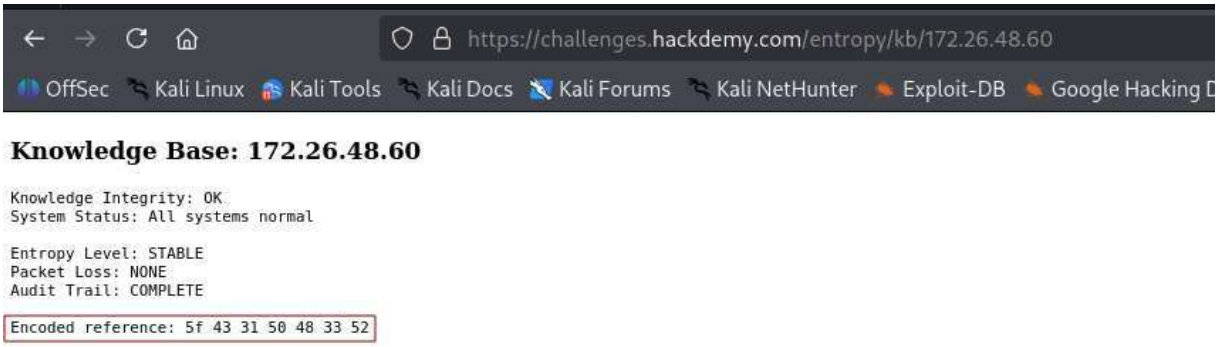
In debugging log report we have a encoded value, we can use cyberchef to decode the value

We have a hint keep looking, we can try to look at the source code of the **host report page**

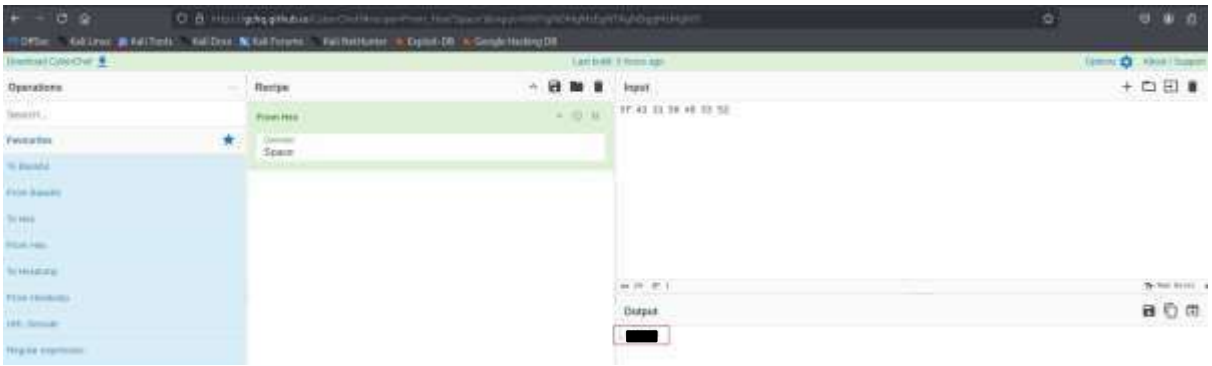


We have retrieved the flag **dead** which is hidden in source code of the host report page is retrieved

The third part **knowledge** hint us to look on the knowledge base of each host



We can see that the host 172.26.48.90 has a encode value, we can use cyberchef to decode it



We have retrieved the third part of the flag, we can gather them and submit it to complete the challenge

CTF{CI[redacted]3R}

PURE LUCK – WALKTHROUGH

DESCRIPTION:

(1/24) * (1/60) * (1/60), flag format:CTF{xxxxxxxxxxxxxxxxxxxxxx}

INTRODUCTION:

This is a simple reverse engineering challenge where we analyze a binary to understand how it evaluates the expression (1/24) * (1/60) * (1/60). Although it looks like a basic math problem, the real task is to inspect how the program computes it internally. By analyzing the main function using radare2, we can determine how the expression is handled and extract the correct flag.

Start the challenge by downloading the challenge file

We can check the file type using '**file'** command

```
pure_luck.out

~/Documents ▸ file pure_luck.out
pure_luck.out: ELF 32-bit LSB executable, Intel i386, version 1 (GNU/Linux), statically linked, BuildID[sha1]=4c882ad6c60813de6112f246b92c0555c00023e2, fo
r GNU/Linux 3.2.0, with debug_info, not stripped
```

We can see it's an executable file we can use the **chmod +x <filename>**

```
Raszzy® ~/Documents ➤ chmod +x pure_luck.out

Raszzy® ~/Documents ➤ ls
pure_luck.out
```

We can see that now the file is executable, we can execute it to check its function

```
Raszzy® ~/Documents ➤ ./pure_luck.out
Pure luck next time :)
```

It looks like It's hiding something, with the description we can try reverse engineering with **radare2 (r2)**

```
Raszzy® ~/Documents ➤ r2 pure_luck.out
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
[0x08049720]>
```

Now, to perform full analysis on the file we can use the '**aaa'** command (analyse all)

```
Raszzy® ~/Documents ➤ r2 pure_luck.out
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
[0x08049720]> aaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Integrate dwarf function information
INFO: Use -AA or aaaa to perform additional experimental analysis
[0x08049720]>
```

We can get into the main function of the executable using the '**s main**' command

```
[0x08049720]> s main
```

Once inside main, the pdf command prints the disassembly of the current function. This shows the assembly instructions, allowing us to understand how the program processes the given expression.

```
[0x08049895]> pdf
            ;-- main:
            ; CODE XREF from loc.__wrap_main 0
┌ 175: int dbg.main (char **argv);
└ - args(sp[0x4..0x4]) vars(29:sp[0x10..0x30])
            0x08049895      8d4c2404        lea ecx, [argv]             ; pure_lock.c:6 ; int main();
            0x08049899      83e4f0          and esp, 0xfffffff0
            0x0804989c      ff71fc          push dword [ecx - 4]
            0x0804989f      55              push ebp
            0x080498a0      89e5            mov ebp, esp
            0x080498a2      53              push ebx
            0x080498a3      51              push ecx
            0x080498a4      83ec20          sub esp, 0x20
            0x080498a7      e898000000      call sym.__x86.get_pc_thunk.ax
            0x080498ac      0548f70900      add eax, 0x9f748
            0x080498b1      c645d843        mov byte [flag], 0x43       ; pure_lock.c:6 ; [redacted] ; 67
            0x080498b5      c645d954        mov byte [var_27h], 0x54    ; pure_lock.c:7 ; [redacted] ; 84
            0x080498b9      c645da46        mov byte [var_26h], 0x46    ; pure_lock.c:8 ; [redacted] ; 70
            0x080498bd      c645db7b        mov byte [var_25h], 0x7b    ; pure_lock.c:9 ; [redacted] ; 123
            0x080498c1      c645dc6e        mov byte [var_24h], 0x6e    ; pure_lock.c:10 ; [redacted] ; 110
            0x080498c5      c645dd30        mov byte [var_23h], 0x30    ; pure_lock.c:11 ; [redacted] ; 48
            0x080498c9      c645de37        mov byte [var_22h], 0x37    ; pure_lock.c:12 ; [redacted] ; 55
            0x080498cd      c645df5f        mov byte [var_21h], 0x5f    ; pure_lock.c:13 ; [redacted] ; 95
            0x080498d1      c645e06c        mov byte [var_20h], 0x6c    ; pure_lock.c:14 ; [redacted] ; 108
            0x080498d5      c645e175        mov byte [var_1fh], 0x75    ; pure_lock.c:15 ; [redacted] ; 117
            0x080498d9      c645e263        mov byte [var_1eh], 0x63    ; pure_lock.c:16 ; [redacted] ; 99
            0x080498dd      c645e36b        mov byte [var_1dh], 0x6b    ; pure_lock.c:17 ; [redacted] ; 107
            0x080498e1      c645e45f        mov byte [var_1ch], 0x5f    ; pure_lock.c:18 ; [redacted] ; 95
            0x080498e5      c645e56a        mov byte [var_1bh], 0x6a    ; pure_lock.c:19 ; [redacted] ; 106
            0x080498e9      c645e675        mov byte [var_1ah], 0x75    ; pure_lock.c:20 ; [redacted] ; 117
            0x080498ed      c645e773        mov byte [var_19h], 0x73    ; pure_lock.c:21 ; [redacted] ; 115
```

We can see that we have retrieved the flag, now we can arrange the flag and submit it

CTF[redacted]is}

LOGGER

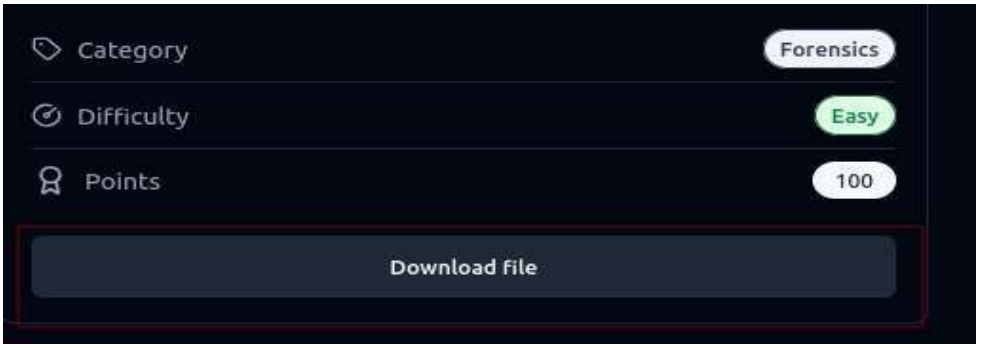
DESCRIPTION:

Sysadmin found an access log that contains some requests to weird endpoints. It's said that the flag lies among them. Are you smart enough to find it?

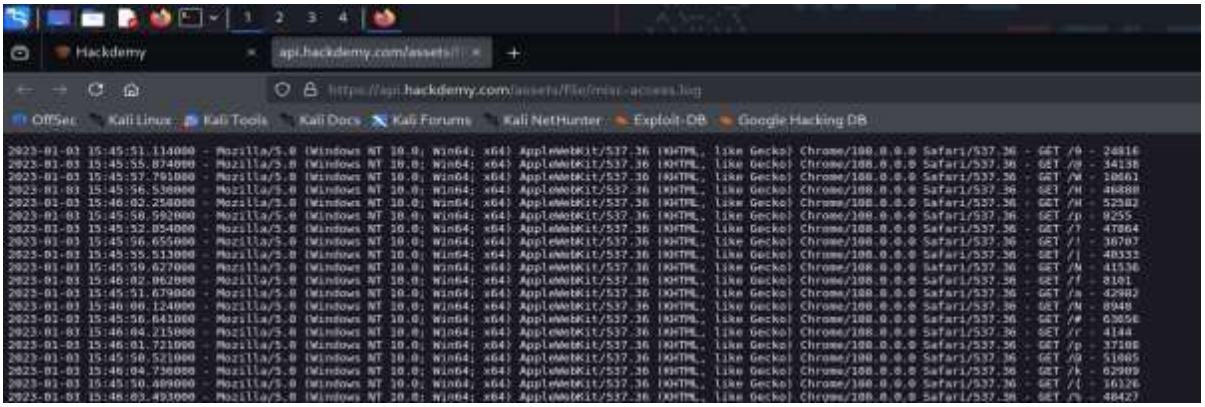
INTRODUCTION:

We have been provided with Active Directory log which is scrambled and position are in wrong order, we should sort it to retrieve the flag

We can download the file using the download button



If the file is not downloaded, we can use the wget command to download from the terminal



```
┌──(root㉿kali)-[/home/kali/Documents]
└─# wget https://api.hackdemy.com/assets/file/misc-access.log
--2026-02-24 03:58:56--  https://api.hackdemy.com/assets/file/misc-access.log
Resolving api.hackdemy.com (api.hackdemy.com)... 65.21.50.139
Connecting to api.hackdemy.com (api.hackdemy.com)|65.21.50.139|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1588614 (1.5M) [text/plain]
Saving to: 'misc-access.log'

misc-access.log                     100%[===================================================>]  1.51M  561KB/s    in 2.8s

2026-02-24 03:59:00 (561 KB/s) - 'misc-access.log' saved [1588614/1588614]
```

Now we can use the sort command with awk to sort the log and separate the flag

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# sort -k1 misc-access.log | awk -F'GET /' '{print $2}' | cut -d' ' -f1 | tr -d '\n'

[jgv(Q%\C4)l\(f/rOp!]ps/S5EKeMHW#-QJ=a(s26aF3t?"2*O?L=xKpRyw^X_(r_$-mH+D(ThAMw$F3r9~4%&B#2^""{(g(Y@r(&1p(7w^u?~|H"??"'!^P5hVU_mh4)]Sb76".Nq3G%E/Vv:#X`anhETxT"g'C}+Y_X4\|rp`),Tg+1|vF(M:!=5[6qE6iHS5sj:D7E=Bv5N/C^"EF}ZjylAlcsHH+8iWPB5C-M&/>@swd3JpPxlllu&EHggaNj>`_O%6hjo:yu*Oo+.0`o#zir3,ny^lO'gleVvcgaG&KkJ_nt{]$zEo=oZ]hiuxuS?cSKKLtL@hyqGd^%8fYa=#Ig9E'Y#!d\!f*Dmlr[vy1~G.l{ev>?[X@BF%1B<O/b$kKdld[$aqYEokw(bi.vt?c(T<|FDEYzSnik0UU((Hqsz3oNgk,78^+t.h^Anar*CvIX)}BrcH2?[_;Brx[*9f\G{L<StABuZ*F<K%#0esv}TU2t\buV\3/J,hbZO=b&6z:H0V++2@W]ms_Lz*-x&TbCn{E\{qANCy3~][(m+paj?Cfga{#HTAJQ^L#PZ3A_X]?7158Vg9CGGw"'e{H{LmSDj>q;(}T!L#(zYz%+q||$*m1[-K3qa$)dkQFU{Y0c#Nl5}H,aP}kWAal}@9Z_V)~r>9JuWlyH\43u.nqR=5|;cGLpHkJY\r%{nNdmTQhx:x2Tdmh+dx/X~%p#9pN"!X1cPX^ufG@uSyu(,Nk6Fw'CPE*`fOsfzZfR8YA}u@H.RiF0pe'u}~:O}!KP=!%_z\G0>vE0|cf*{Jk,CRDi^B\J0M~^#OuP[&r#FpPpH}51ggta8Nxj2e5H$H2s+_MRRQ7q@b:T$$%W"]K-)JyU1`<VXfU(^aX"9HViK|2Ahs;6h$iqtM#h/a9It&\6!CTF{[redacted]B}
```

CT[redacted]n3}

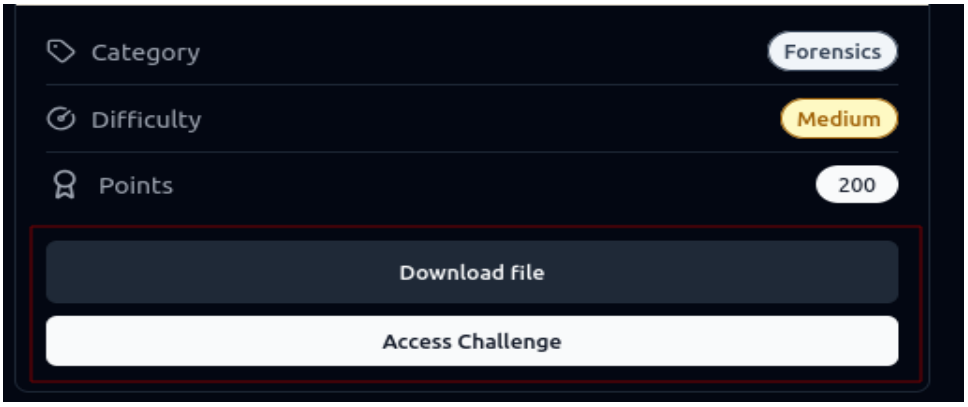
Detection Engineering Foundations (YARA)

DESCRIPTION:

This lab simulates a real-world detection engineering task: identifying malicious indicators within large datasets using YARA. Participants will develop rules that account for encoding, obfuscation, and contextual logic to reliably detect targeted artifacts. The focus is on building resilient signatures rather than relying on direct inspection.

INTRODUCTION:

We can download the required files using the download button and access the web page using the access challenge button



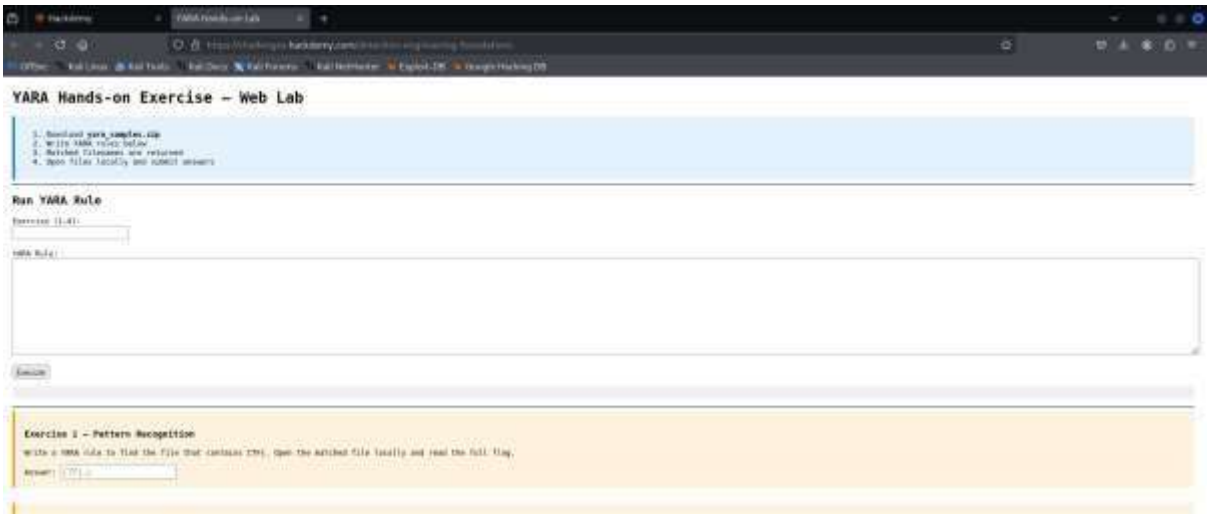
We have given a zip file, we can try to extract it

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
sample-files.zip
```

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# unzip sample-files.zip
Archive:  sample-files.zip
```

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
sample-files.zip  yara_samples_final

┌──(root㉿kali)-[/home/kali/Documents]
└─# cd yara_samples_final

┌──(root㉿kali)-[/home/kali/Documents/yara_samples_final]
└─# cd samples

┌──(root㉿kali)-[/home/kali/Documents/yara_samples_final/samples]
└─# ls
Exercise1  Exercise2  Exercise3  Exercise4

┌──(root㉿kali)-[/home/kali/Documents/yara_samples_final/samples]
└─# cd Exercise1

┌──(root㉿kali)-[/home/…/Documents/yara_samples_final/samples/Exercise1]
└─# ls
ahpcyzcn.txt  dowxvdcz.txt  exrdbspp.txt  lejflnoc.txt  nkciymos.txt  qvcjayfw.txt  vcfsmcql.txt  yenydikm.txt
csphhyca.txt  duqloaab.txt  gpbqhkgc.txt  lgqcnvnk.txt  nyrqoyhs.txt  rgnwpywg.txt  vkgfkykn.txt  zxfluyop.txt
ctbqollq.txt  ebhixxce.txt  jyixqdna.txt  nibeojnf.txt  qpqfdcpk.txt  skdhhstl.txt  xtonnpbn.txt
```

We can see that there are four exercise files with each containing 20+ files, we can use the web page and use **yara rules** to find the file with the flag



We can see that we have a input field to enter the exercise number and a input field to write yara rule to find the file with flag

Exercise 1 – Pattern Recognition

Write a YARA rule to find the file that contains CTF{. Open the matched file locally and read the full flag.

Run YARA Rule

Exercise (1–4):

1

YARA Rule:

```
rule Pattern_Recognition
{
 strings:
   $flag = "CTF{"

 condition:
   $flag
}
```

Execute

Matched files:

- [redacted]

We can open the file in terminal to retrieve the flag in the respective directory

```
┌──(root㉿kali)-[/home/kali/Documents/yara_samples_final/samples]
└─# cd Exercise1

┌──(root㉿kali)-[/home/…/Documents/yara_samples_final/samples/Exercise1]
└─# ls
ahpcyzcn.txt  dowxvdcz.txt  exrdbspp.txt  lejflnoc.txt  nkciymos.txt  qvcjayfw.txt  vcfsmcql.txt  yenydikm.txt
csphhyca.txt  duqloaab.txt  gpbqhkgc.txt  lgqcnvnk.txt  nyrqoyhs.txt  rgnwpywg.txt  vkgfkykn.txt  zxfluyop.txt
ctbqollq.txt  ebhixxce.txt  jyixqdna.txt  nibeojnf.txt  qpqfdcpk.txt  skdhhstl.txt  xtonnpbn.txt

┌──(root㉿kali)-[/home/…/Documents/yara_samples_final/samples/Exercise1]
└─# cat vcfsmcql.txt
[redacted]
```

Exercise 2 – Multi-Condition Logic

Find the file containing both strings:

"Yet another"

"Ridiculous acronym"

Run YARA Rule

Exercise (1–4):

2

YARA Rule:

```
rule Pattern_Recognition
{
 strings:
   $first = "Yet another"
   $second = "Ridiculous acronym"

 condition:
   $first and $second
}
```

Execute

Matched files:

- [redacted]

Here we are asked the file name as the answer, so the answer is mŋxqgsx.txt

Exercise 3 – Base64 Detection

Find the file containing a Base64 string starting with Q1RG.

Run YARA Rule

Exercise (1–4):

3

YARA Rule:

```
rule Pattern_Recognition
{
 strings:
   $base64 = "Q1RG"

 condition:
   $base64
}
```

Execute

Matched files:

- [redacted]

Here we are asked the file name as the answer, so the answer is hnntxhtp.txt

Exercise 4 – XOR Encryption Analysis

Determine the XOR key and find the encrypted string.

Here the task is to determine the XOR key ourself we can try random XOR values like {0x01, 0x02, 0x03}

Run YARA Rule

Exercise (1–4):

4

YARA Rule:

```
rule Pattern_Recognition
{
 strings:
   $x0r = "0x01"

 condition:
   $x0r
}
```

Execute

Matched files:

- [redacted]

We can use another method that will brute force ("CTF{" xor tells YARA to try all possible single-byte XOR keys

It matches if the file contains a XOR-encrypted version of CTF{

Run YARA Rule

Exercise (1–4):

4

YARA Rule:

```
rule Pattern_Recognition
{
 strings:
    $x0r = "CTF" xor

 condition:
    $x0r
}
```

Execute

Matched files:

- [redacted]

Now we can cat the file to look at the content

```
┌──(root㉿kali)-[/home/kali/Documents/yara_samples_final/samples]
└─# cd Exercise4

┌──(root㉿kali)-[/home/…/Documents/yara_samples_final/samples/Exercise4]
└─# cat mnjladvr.txt
This file contains XOR encrypted data.
[redacted]
```

Now we have all the four exercises answer we can submit them to retrieve the final flag to complete the challenge

Exercise 1 — Pattern Recognition

Write a YARA rule to find the file that contains CTF{. Open the matched file locally and read the full flag.

Answer: [redacted]

Exercise 2 — Multi-Condition Logic

Find the file containing **both** strings:

- "Yet another"
- "Ridiculous acronym"

Answer (filename): [redacted]

Exercise 3 — Base64 Detection

Find the file containing a Base64 string starting with Q1RG.

Answer (filename): [redacted]

Exercise 4 — XOR Encryption Analysis

Determine the XOR key and find the encrypted string.

XOR Key (hex): [redacted]

Encrypted Stri[redacted]

Validate Answers

All correct!

[redacted]

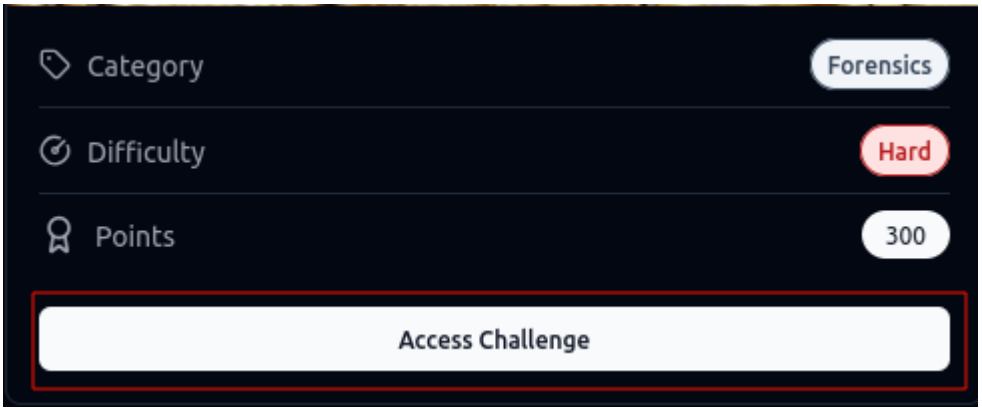
RESUIDE

DESCRIPTION:

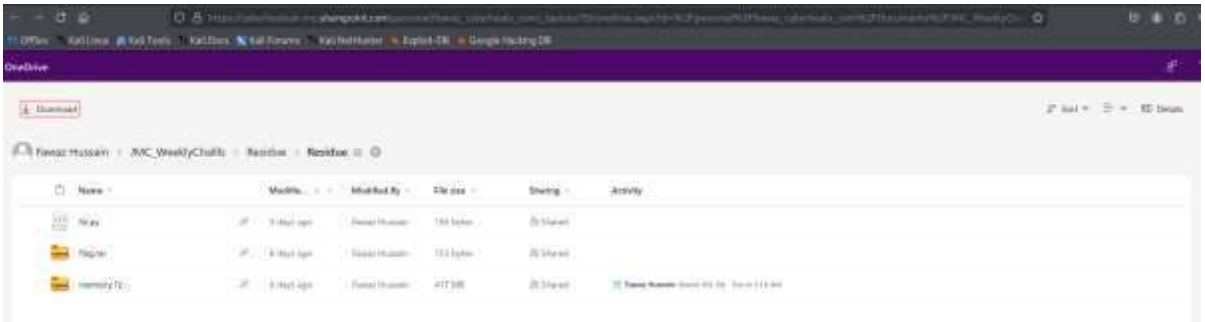
A Windows 7 memory dump was recovered from a system suspected of hiding sensitive data. The user was actively browsing and working with encrypted files before the capture. The flag is split into two parts. Everything required to reconstruct the complete flag exists within the memory dump. Find the seed and you shall reap the PAYLOAD.

INTRODUCTION:

we can use the access challenge button to get into the download page



We are redirected to the onedrive page from where we can download the challenge files



We can see that we have the file downloaded, we can take look at its content and we can unzip the zip files

```
Raszzy㉿~/Documents ➤ ls
fix.py  flag.rar  memory.7z
```

We can see that the memory.7z gives us with a memory.raw file which can be analysed using volataility

```
Raszzy㉿~/Documents ➤ 7z e memory.7z

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
 64-bit locale=en_US.UTF-8 Threads:32 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 436874450 bytes (417 MiB)

Extracting archive: memory.7z
--
Path = memory.7z
Type = 7z
Physical Size = 436874450
Headers Size = 130
Method = LZMA2:25
Solid = -
Blocks = 1

Everything is Ok

Size:       2147483648
Compressed: 436874450
```

```
Raszzy㉿~/Documents ➤ ls
fix.py  flag.rar  memory.7z  memory.raw
```

We can now try to unzip the flag.rar it looks like It contains the flag.txt but we don't have the password right now

```
Raszzy㉿~/Documents ➤ rar e flag.rar

RAR 7.12   Copyright (c) 1993-2025 Alexander Roshal   23 Jun 2025
Trial version             Type 'rar -?' for help

Extracting from flag.rar

Enter password (will not be echoed) for flag.txt:
```

We can now download volataility to analyze the file, for installing volatility we should first create a python virtual environment, we can download python enev (environment)

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# sudo apt install python3-venv
python3-venv is already the newest version (3.13.7-1+b1).
python3-venv set to manually installed.
The following package was automatically installed and is no longer required:
  libjs-underscore
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2356
```

Now we can create a directory or a space for the virtual environment

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# python3 -m venv volatile

┌──(root㉿kali)-[/home/kali/Documents]
└─# ls
fix.py  flag.rar  memory.7z  memory.raw  volatile
```

```
┌──(root㉿kali)-[/home/kali/Documents]
└─# source volatile/bin/activate

┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─#
```

We have created and activated the virtual environment for python now we can install volatility

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# pip install volatility3
Collecting volatility3
  Downloading volatility3-2.27.0-py3-none-any.whl.metadata (7.5 kB)
Collecting pefile ≥ 2024.8.26 (from volatility3)
  Downloading pefile-2024.8.26-py3-none-any.whl.metadata (1.4 kB)
Downloading volatility3-2.27.0-py3-none-any.whl (1.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 1.5 MB/s  0:00:01
Downloading pefile-2024.8.26-py3-none-any.whl (74 kB)
Installing collected packages: pefile, volatility3
Successfully installed pefile-2024.8.26 volatility3-2.27.0
```

Now we can start analysing the memory.raw file

By running the command **“vol -f memory.raw windows.info”** we can gathers the information about system the raw file was taken from

```
Variable        Value

Kernel Base     0xf80002806000
DTB     0x187000
Symbols file:///home/kali/Documents/volatile/lib/python3.13/site-packages/volatility3/symbols/windows/ntkrnlmp.pdb/3844DBB920174967BE7AA4A2C20
A30FA-2.json.xz
Is64Bit True
IsPAE   False
layer_name      0 WindowsIntel32e
memory_layer    1 FileLayer
KdDebuggerDataBlock     0xf800029f70a0
NTBuildLab      7601.17514.amd64fre.win7sp1_rtm.
CSDVersion      1
KdVersionBlock  0xf800029f7068
Major/Minor     15.7601
MachineType     34404
KeNumberProcessors      1
SystemTime      2026-02-15 13:00:42+00:00
NtSystemRoot    C:\Windows
NtProductType   NtProductWinNt
NtMajorVersion  6
NtMinorVersion  1
PE MajorOperatingSystemVersion  6
PE MinorOperatingSystemVersion  1
PE Machine      34404
PE TimeDateStamp        Sat Nov 20 09:30:02 2010
```

We can see it is a windows system running version 7

Now we can try to list the process from the raw file using the command "**vol -f memory.raw windows.pslist"**

```
1804    444     svchost.exe     0xfa80032adb30  14      233     0       False   2026-02-15 10:28:29.000000 UTC  N/A     Disabled
2536    444     sppsvc.exe      0xfa8002450b30  4       148     0       False   2026-02-15 10:30:23.000000 UTC  N/A     Disabled
2392    444     svchost.exe     0xfa8002490b30  12      315     0       False   2026-02-15 10:30:23.000000 UTC  N/A     Disabled
2380    1104    chrome.exe      0xfa8001911560  26      1003    1       False   2026-02-15 12:39:02.000000 UTC  N/A     Disabled
1660    2380    chrome.exe      0xfa8003312060  7       103     1       False   2026-02-15 12:39:03.000000 UTC  N/A     Disabled
720     2380    chrome.exe      0xfa8003035060  2       55      1       False   2026-02-15 12:39:11.000000 UTC  N/A     Disabled
2300    2380    chrome.exe      0xfa80022539d0  13      308     1       False   2026-02-15 12:39:19.000000 UTC  N/A     Disabled
2312    2380    chrome.exe      0xfa800329cb30  8       232     1       False   2026-02-15 12:40:00.000000 UTC  N/A     Disabled
2824    576     WmiPrvSE.exe    0xfa800320cb30  6       201     0       False   2026-02-15 12:40:01.000000 UTC  N/A     Disabled
2332    2380    chrome.exe      0xfa80033c9630  13      190     1       False   2026-02-15 12:40:30.000000 UTC  N/A     Disabled
1812    2380    chrome.exe      0xfa8003424b30  16      296     1       False   2026-02-15 12:40:48.000000 UTC  N/A     Disabled
2044    2380    chrome.exe      0xfa8003427060  12      169     1       False   2026-02-15 12:40:52.000000 UTC  N/A     Disabled
2620    2380    chrome.exe      0xfa8003424060  13      190     1       False   2026-02-15 12:40:52.000000 UTC  N/A     Disabled
1656    2380    chrome.exe      0xfa8003433b30  14      316     1       False   2026-02-15 12:40:56.000000 UTC  N/A     Disabled
604     1104    cmd.exe 0xfa8003477b30  1       19      1       False   2026-02-15 12:41:13.000000 UTC  N/A     Disabled
2616    360     conhost.exe     0xfa800344bb30  2       51      1       False   2026-02-15 12:41:13.000000 UTC  N/A     Disabled
3064    2380    chrome.exe      0xfa80034a2b30  12      212     1       False   2026-02-15 12:41:17.000000 UTC  N/A     Disabled
2972    2380    chrome.exe      0xfa80034d7060  10      195     1       False   2026-02-15 12:41:22.000000 UTC  N/A     Disabled
808     1104    notepad.exe     0xfa800316ca30  1       61      1       False   2026-02-15 12:42:57.000000 UTC  N/A     Disabled
2524    2380    chrome.exe      0xfa800352a3b0  13      208     1       False   2026-02-15 12:43:38.000000 UTC  N/A     Disabled
1240    816     taskeng.exe     0xfa8003508610  4       79      0       False   2026-02-15 12:58:49.000000 UTC  N/A     Disabled
1156    2380    chrome.exe      0xfa80019dc060  8       100     1       False   2026-02-15 12:59:21.000000 UTC  N/A     Disabled
2460    736     audiodg.exe     0xfa8002497270  4       127     0       False   2026-02-15 12:59:43.000000 UTC  N/A     Disabled
1396    1104    WinRAR.exe      0xfa8003504920  4       203     1       False   2026-02-15 12:59:54.000000 UTC  N/A     Disabled
1908    1104    DumpIt.exe      0xfa8003524b30  5       46      1       True    2026-02-15 13:00:13.000000 UTC  N/A     Disabled
3028    360     conhost.exe     0xfa800330f7b0  2       50      1       False   2026-02-15 13:00:14.000000 UTC  N/A     Disabled
```

First create a directory to save the content

With the hint and process we can see that it looks like chrome.exe has been used more we can grep only the chrome.exe process and save it in separate directory

First create a directory to save the content using **"mkdir <directory name>"**

```
0xf900ce75c000  0x45f06000      0x1000  0x22de1000      pid.2380.dmp
0xf900ce75d000  0x57e07000      0x1000  0x22de2000      pid.2380.dmp
0xf900ce75e000  0x36808000      0x1000  0x22de3000      pid.2380.dmp
0xf900ce75f000  0x9809000       0x1000  0x22de4000      pid.2380.dmp
0xf900ce760000  0xf40a000       0x1000  0x22de5000      pid.2380.dmp
0xf900ce761000  0x4d90b000      0x1000  0x22de6000      pid.2380.dmp
0xf900ce762000  0x7360c000      0x1000  0x22de7000      pid.2380.dmp
0xf900ce763000  0x5c80d000      0x1000  0x22de8000      pid.2380.dmp
0xf900ce764000  0x1770e000      0x1000  0x22de9000      pid.2380.dmp
0xf900ce765000  0x7730f000      0x1000  0x22dea000      pid.2380.dmp
0xf900ce766000  0xdb10000       0x1000  0x22deb000      pid.2380.dmp
0xf900ce767000  0x34911000      0x1000  0x22dec000      pid.2380.dmp
0xf900ce768000  0x51412000      0x1000  0x22ded000      pid.2380.dmp
0xf900ce769000  0x34d13000      0x1000  0x22dee000      pid.2380.dmp
0xf900ce76a000  0x57914000      0x1000  0x22def000      pid.2380.dmp
0xf900ce76b000  0x4be15000      0x1000  0x22df0000      pid.2380.dmp
0xf900ce76c000  0x33916000      0x1000  0x22df1000      pid.2380.dmp
0xf900ce76d000  0x65817000      0x1000  0x22df2000      pid.2380.dmp
```

using the command “**vol -f memory.raw -o chrome_dum windows.memmap –pid 2380 –dump”**

we can see that we have the dump file in the directory we created

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents/chrome_dum]
└─# ls
pid.2380.dmp
```

Now using the hint we can try to take look at two values **seed** and **PAYLOAD**

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# strings memory.raw | grep PAYLOAD
IKEEXT_CERT_FLAG_DISABLE_REQUEST_PAYLOAD
const INPUT_PAYLOAD: &str = "q255eZh-AMESYYCjCxWyzJIKvCjn5GrpBQowaffpZVH";
    println!("    {}", INPUT_PAYLOAD);
    let normalized = normalize(INPUT_PAYLOAD);
CONTROL_PAYLOAD_TOO_LARGE
OVERSIZED_PAYLOAD
```

We tried using the cyberchef and other things but nothing worked, Now using the strings command, we can look at the content of the dump

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents/chrome_dum]
└─# strings pid.2380.dmp
Actx
SsHd,
GsHd(
GsHd(
GsHd(
GsHd(
Actx
[IY-
SsHd,
[IY-D
SsHd,
SsHd,
\a*p
GsHd(
GsHd(
GsHd(
SsHd,
GsHd(
```

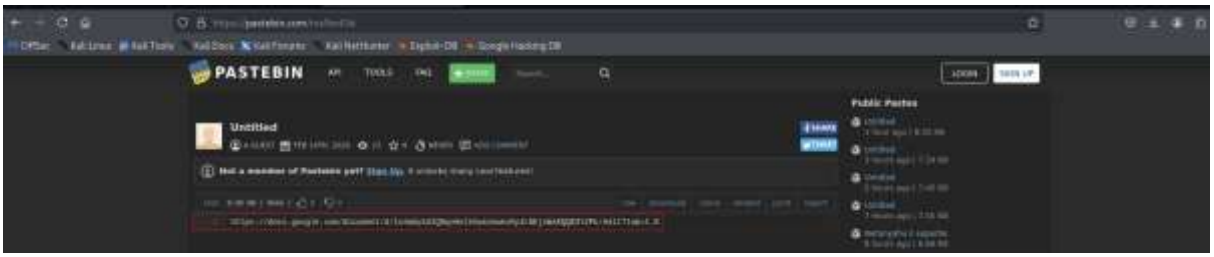
We can see it produce lot of result, by using the hint we know we should take a look at the seed so we can use the command “strings pid.2380.dmp | grep “seed”

```
Networking-MPSSVC-Rules-EnterpriseEdition
Microsoft-Windows-EnterpriseEdition~31bf3856ad364e35~amd64~~6.1.7601.17514
Microsoft-Windows-EnterpriseEdition~31bf3856ad364e35~amd64~~6.1.7601.17514
Microsoft-Windows-EnterpriseEdition~31bf3856ad364e35~amd64~~6.1.7601.17514
        '{"seed":"0×37","blob":"X0NDR0QNGBhHVkRDUlVeWRlUWFoYRVJEDkADBFM="}'
  "prompt_seed": "20230123ulXSkgts",
"seed":"
97,"seed
{"seed":"0×37","blob":"X0NDR0QNGBhHVkRDUlVeWRlUWFoYRVJEDkADBFM="}
{"seed":"0×37","blob":"X0ND
```

It looks like there is an encode value with hex key we can use cyber chef to decode that

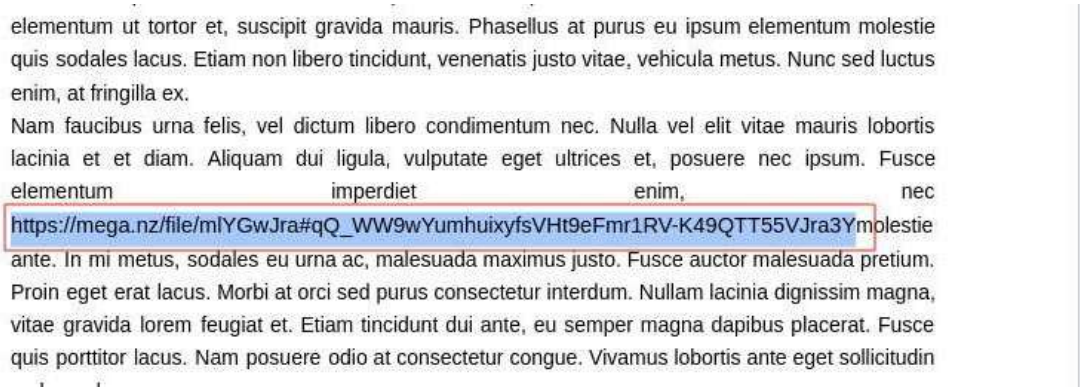
It automatically found the base64 and decoded but we don’t have any clear text, we have a key 0x37 we can use it on xor

We got a Pastebin link lets open it to look at its content



On opening the Pastebin link we got another link, lets open it to look at its content

We have a file named hidden among noise, we can analyse the file further to retrieve some information



elementum ut tortor et, suscipit gravida mauris. Phasellus at purus eu ipsum elementum molestie quis sodales lacus. Etiam non libero tincidunt, venenatis justo vitae, vehicula metus. Nunc sed luctus enim, at fringilla ex.

Nam faucibus urna felis, vel dictum libero condimentum nec. Nulla vel elit vitae mauris lobortis lacinia et et diam. Aliquam dui ligula, vulputate eget ultrices et, posuere nec ipsum. Fusce elementum imperdiet enim, nec https://mega.nz/file/mlYGwJra#qQ_WW9wYumhuixyfsVHt9eFmr1RV-K49QTT55VJra3Ymolestie ante. In mi metus, sodales eu urna ac, malesuada maximus justo. Fusce auctor malesuada pretium. Proin eget erat lacus. Morbi at orci sed purus consectetur interdum. Nullam lacinia dignissim magna, vitae gravida lorem feugiat et. Etiam tincidunt dui ante, eu semper magna dapibus placerat. Fusce quis porttitor lacus. Nam posuere odio at consectetur congue. Vivamus lobortis ante eget sollicitudin

We have found another link; it looks like it a mega link that contain some files we can open and take a look at its content

It has a zip file named clue.7z, we can download and unzip to take a look at its contents

```
┌──(volatile)-(root㉿kali)-[/home/kali/Documents]
└─# ls
chrome_dum  clue.7z  fix.py  flag.rar  memory.7z  memory.raw  vol3env  volatile

┌──(volatile)-(root㉿kali)-[/home/kali/Documents]
└─# 7z e clue.7z

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
 64-bit locale=en_US.UTF-8 Threads:32 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 234 bytes (1 KiB)

Extracting archive: clue.7z
--
Path = clue.7z
Type = 7z
Physical Size = 234
Headers Size = 138
Method = LZMA2:12 7zAES
Solid = -
Blocks = 1


Enter password (will not be echoed):
```

We have downloaded the zip file and it prompt for password when we try to extract them

We can look at the same hidden among noise file for the password (we should search for a meaningful word among the noise)

In auctor mauris tincidunt est aliquet, id lobortis purus commodo. Nam sit amet lectus sit amet ex efficitur tempus. Morbi eleifend, massa ut "One half reflects, the other rotates.", ac scelerisque ligula odio sit amet quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Sed aliquam dapibus ligula vel porta. Donec in nisl volutpat, vestibulum sapien eget, accumsan urna. Mauris sagittis mauris sit amet tristique finibus.

Now, if we remember the PAYLOAD we retrieved, we can use the hint on the PAYLOAD

We can use the hint on AI to find a programme that will do the task

"One half reflects, the other rotates."
That usually means:

- **First half → Reflect** (reverse the string)
- **Second half → Rotate** (likely Caesar cipher / ROT-style shift)

```
s='q255eZh-AMESYYCjCxWyzJIKvCjn5GrpBQowaffpZVH"'.strip('"')
m=len(s)//2
a=s[:m][::-1]
b=s[m:]
```

```
def rot13(x):
return ''.join(
chr((ord(c)-65+13)%26+65) if c.isupper() else
chr((ord(c)-97+13)%26+97) if c.islower() else c
for c in x
)

print(a + rot13(b))
```

We can run this python code to retrieve the password for the file clue.7z

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# nano script.py

┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# python3 script.py
zyWxCjCYYSEMA-hZe552qWVXiPwa5TecODbjnsscMIU
```

We can try this password to extract the zip file

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# 7z e clue.7z

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
 64-bit locale=en_US.UTF-8 Threads:32 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 234 bytes (1 KiB)

Extracting archive: clue.7z
--
Path = clue.7z
Type = 7z
Physical Size = 234
Headers Size = 138
Method = LZMA2:12 7zAES
Solid = -
Blocks = 1


Enter password (will not be echoed):
Everything is Ok

Size:       89
Compressed: 234
```

The password has worked an we got a file named clue.png, lets try to open it

It looks like the png file is corrupted we can to read the content of the fix.py and execute it to fix the png file

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# cat fix.py
import zlib

d = open("file","rb").read()
i = next(i for i in range(len(d)) if d[i]==0×78 and d[i+1] in (1,156,218))
print(zlib.decompress(d[i:]).decode())
```

We can see that we have to modify this program to fix the png file, we can use nano to modify the file

```
File  Actions  Edit  View  Help
  GNU nano 8.4                                                    fix.py *
import zlib

d = open("clue.png","rb").read()
i = next(i for i in range(len(d)) if d[i]==0×78 and d[i+1] in (1,156,218))
print(zlib.decompress(d[i:]).decode())
```

We have modified the program, now we can try running it

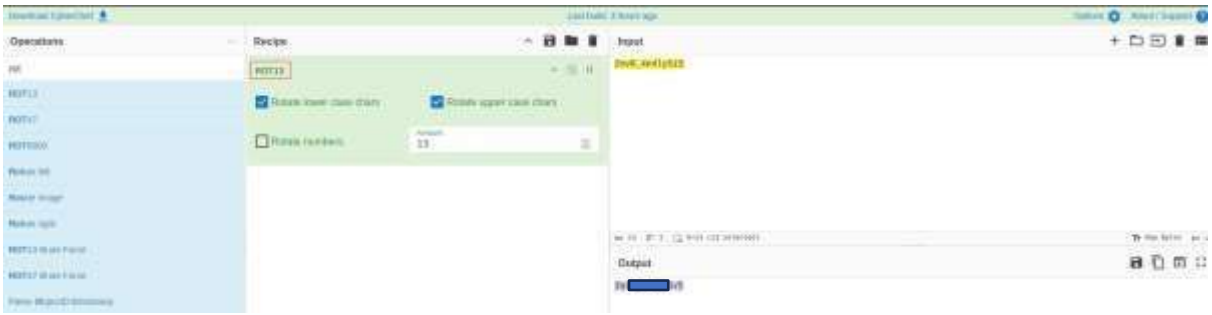
```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# python3 fix.py
CTF{h[redacted]s_
```

We have retrieved the first half of the flag

Now using the second hint we can try to get the environment variable

```
┌──(volatile)─(root㉿kali)-[/home/kali/Documents]
└─# vol -f memory.raw windows.envars | grep RAR
604gresscmd.exe00×301dd0        RAR_INFO        ROT13
604     cmd.exe 0×301dd0        RAR_PASS        3nvR_4n4ly5i5
1396    WinRAR.exe      0×311e00        ALLUSERSPROFILE C:\ProgramData
1396    WinRAR.exe      0×311e00        APPDATA C:\Users\Guy\AppData\Roaming
1396    WinRAR.exe      0×311e00        CommonProgramFiles      C:\Program Files\Common Files
```

We can see that the environment variable contains remaining flag, but it doesn't work (if we notice clearly, we can see the above ROT13 value so we can put the value in cyberchef with ROT13)



Now we can try combining both the part to make sure the flag is correctly retrieved

CTF{h3[redacted]y5i5}

EVAPORATED

DESCRIPTION:

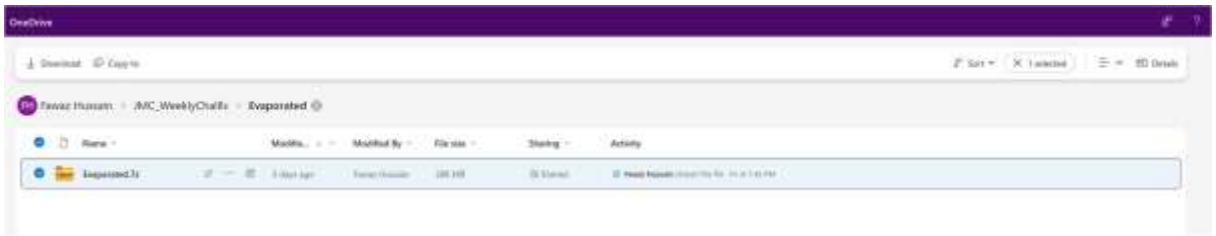
A Windows 7 memory image was acquired shortly after suspected malicious activity. Evidence suggests scripted execution followed by persistence, staging, and a brief cleanup attempt. References to an updater-style component, collected artifacts, and a clean temporary variable were observed during preliminary analysis. Some data may reside in native Windows text formats rather than standard ASCII. Analyze the memory to reconstruct the attacker's actions and recover the complete flag.

INTRODUCTION:

We have to use the volataility tool to analyse the given raw file to discover the hidden flag

We can access the challenge using the access challenge button

We are redirected to a onedrive page from where we can download the challenge file



We have a 7z zip file named evaporated.7z we can unzip it

```
┌──(sibhath㉿sibhath)-[~/Documents]
└─$ 7z e Evaporated.7z

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
 64-bit locale=en_US.UTF-8 Threads:5 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 302472705 bytes (289 MiB)

Extracting archive: Evaporated.7z
--
Path = Evaporated.7z
Type = 7z
Physical Size = 302472705
Headers Size = 148
Method = LZMA:23
Solid = -
Blocks = 1

Everything is Ok

Size:       2147483648
Compressed: 302472705

┌──(sibhath㉿sibhath)-[~/Documents]
└─$ ls
Evaporated.7z  memory.raw
```

We got a memory.raw file we can try to analyse it using the volataility tool

Before that we have to download volataility by setting a python virtual environment

```
┌──(sibhath㉿sibhath)-[~/Documents]
└─$ python3 -m venv volatile

┌──(sibhath㉿sibhath)-[~/Documents]
└─$ ls
Evaporated.7z  memory.raw  volatile

┌──(sibhath㉿sibhath)-[~/Documents]
└─$ source volatile/bin/activate
```

Now we can use pip to install volataility command **“pip install volataility3”**

```
┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ pip install volatility3
Collecting volatility3
  Using cached volatility3-2.27.0-py3-none-any.whl.metadata (7.5 kB)
Collecting pefile ≥ 2024.8.26 (from volatility3)
  Using cached pefile-2024.8.26-py3-none-any.whl.metadata (1.4 kB)
Using cached volatility3-2.27.0-py3-none-any.whl (1.4 MB)
Using cached pefile-2024.8.26-py3-none-any.whl (74 kB)
Installing collected packages: pefile, volatility3
Successfully installed pefile-2024.8.26 volatility3-2.27.0
```

We have the virtual environment activated and the volatility installed lets now analyse the file

To get the info about the system we can run the following command **“vol -f memory.raw windows.info”**

```
┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ vol -f memory.raw windows.info
Volatility 3 Framework 2.27.0
Progress:  100.00               PDB scanning finished
Variable        Value

Kernel Base     0xf800285f000
DTB     0x187000
Symbols file:///home/sibhath/Documents/volatile/lib/python3.13/site-packages/volatility3/symbols/windows/ntkrnlmp.pdb/3B46DBB92817496 7BE7AA4A2C2063BFA-2.json.xz
Is64Bit True
IsPAE   False
layer_name      0 WindowsIntel32e
memory_layer    1 FileLayer
KdDebuggerDataBlock     0xf80002a500a0
NTBuildLab      7601.17514.amd64fre.win7sp1_rtm.
CSDVersion      1
KdVersionBlock  0xf80002a5008
Major/Minor     15.7601
MachineType     34404
KeNumberProcessors      1
SystemTime      2026-02-27 16:55:06+00:00
NtSystemRoot    C:\Windows
NtProductType   NtProductWinNt
NtMajorVersion  6
NtMinorVersion  1
PE MajorOperatingSystemVersion  6
PE MinorOperatingSystemVersion  1
PE Machine      34404
PE TimeDateStamp        Sat Nov 20 09:30:02 2010
```

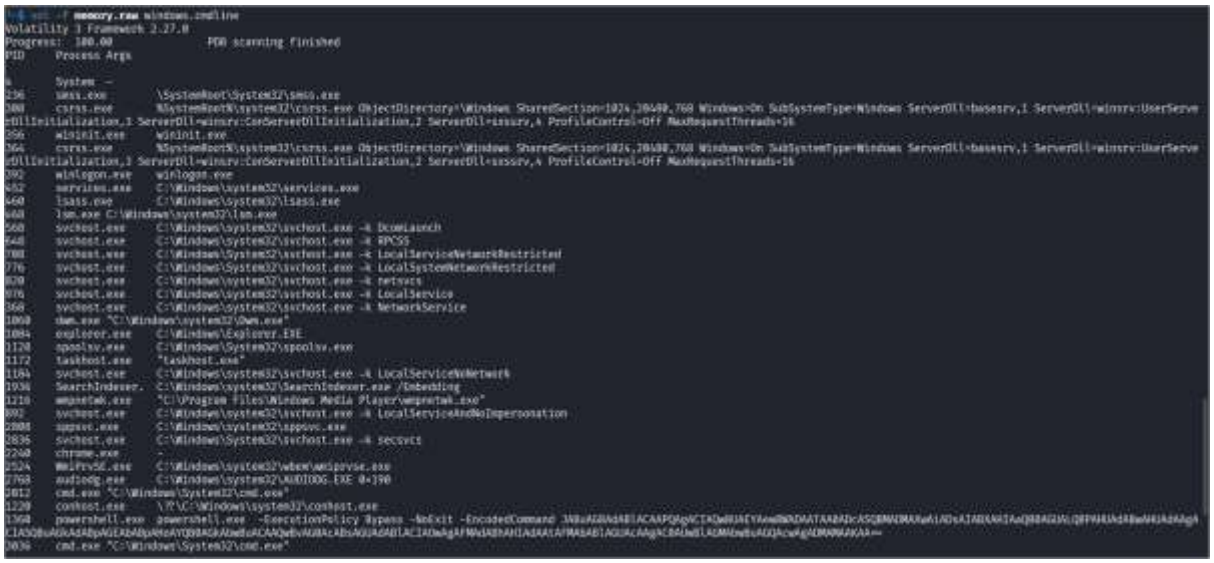
We can see it is a windows machine running windows 7 and also other information’s

Now we can try to list out the processes using the command **“vol -f memory.raw windows.pslist”**

```
┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ vol -f memory.raw windows.pslist
Volatility 3 Framework 2.27.0
Progress:  100.00               PDB scanning finished
PID     PPID    ImageFileName   Offset(V)       Threads Handles SessionId       Wow64   CreateTime      ExitTime        File output

4       0       System  0xfa8018b3b30   78      510     N/A     False   2026-02-27 15:22:54.000000 UTC  N/A     Disabled
236     4       smss.exe        0xfa8001fa0310  2       29      N/A     False   2026-02-27 15:22:54.000000 UTC  N/A     Disabled
308     292     csrss.exe       0xfa8002fc9060  9       386     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
356     292     wininit.exe     0xfa8018d3530   3       75      0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
364     348     csrss.exe       0xfa8019bd160   9       252     1       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
392     348     winlogon.exe    0xfa8002b40060  4       116     1       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
452     356     services.exe    0xfa8002c8e490  8       187     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
460     356     lsass.exe       0xfa8002ca0b30  7       542     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
468     356     lsm.exe 0xfa8002ca8060  10      149     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
568     452     svchost.exe     0xfa8002d91060  9       350     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
648     452     svchost.exe     0xfa8002d84060  7       251     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
708     452     svchost.exe     0xfa8002da97a0  22      498     0       False   2026-02-27 15:22:55.000000 UTC  N/A     Disabled
776     452     svchost.exe     0xfa8002d8f9c0  16      366     0       False   2026-02-27 15:22:56.000000 UTC  N/A     Disabled
820     452     svchost.exe     0xfa8002decb30  29      918     0       False   2026-02-27 15:22:56.000000 UTC  N/A     Disabled
976     452     svchost.exe     0xfa8002e59740  20      474     0       False   2026-02-27 15:22:56.000000 UTC  N/A     Disabled
368     452     svchost.exe     0xfa8002e9cb30  16      458     0       False   2026-02-27 15:22:56.000000 UTC  N/A     Disabled
1060    776     dwm.exe 0xfa8002ef5910  3       70      1       False   2026-02-27 15:22:58.000000 UTC  N/A     Disabled
1084    1040    explorer.exe    0xfa8003f86700  43      1168    1       False   2026-02-27 15:22:58.000000 UTC  N/A     Disabled
1124    452     spoolsv.exe     0xfa8003f40060  13      261     0       False   2026-02-27 15:22:56.000000 UTC  N/A     Disabled
1172    452     taskhost.exe    0xfa8002f63b30  9       188     1       False   2026-02-27 15:22:57.000000 UTC  N/A     Disabled
1184    452     svchost.exe     0xfa8002f625f0  17      300     0       False   2026-02-27 15:22:57.000000 UTC  N/A     Disabled
1936    452     SearchIndexer.  0xfa8002b08060  13      620     0       False   2026-02-27 15:23:03.000000 UTC  N/A     Disabled
1136    452     wmpnetwk.exe    0xfa8003ca6a0   8       221     0       False   2026-02-27 15:23:03.000000 UTC  N/A     Disabled
892     452     svchost.exe     0xfa8003336b30  14      331     0       False   2026-02-27 15:23:04.000000 UTC  N/A     Disabled
2008    452     sppsvc.exe      0xfa8001b80060  7       150     0       False   2026-02-27 15:24:58.000000 UTC  N/A     Disabled
2836    452     svchost.exe     0xfa8002b0a630  13      320     0       False   2026-02-27 15:24:58.000000 UTC  N/A     Disabled
2240    1884    chrome.exe      0xfa8003a55d0   8       -       1       False   2026-02-27 16:51:26.000000 UTC  2026-02-27 16:54:42.000000 UTC  Disabled
2524    568     WmiPrvSE.exe    0xfa8003b6c630  11      206     0       False   2026-02-27 16:51:39.000000 UTC  N/A     Disabled
2760    708     audiodg.exe     0xfa8003e85060  6       131     0       False   2026-02-27 16:52:28.000000 UTC  N/A     Disabled
2012    1884    cmd.exe 0xfa8003a29750  1       22      1       False   2026-02-27 16:52:30.000000 UTC  N/A     Disabled
1220    364     conhost.exe     0xfa8003a360060 2       53      1       False   2026-02-27 16:52:30.000000 UTC  N/A     Disabled
1360    2012    powershell.exe  0xfa8003738b50  5       289     1       False   2026-02-27 16:52:42.000000 UTC  N/A     Disabled
3036    1884    cmd.exe 0xfa8003a50b30  1       23      1       False   2026-02-27 16:52:49.000000 UTC  N/A     Disabled
1656    364     conhost.exe     0xfa8003a4e060  2       51      1       False   2026-02-27 16:52:49.000000 UTC  N/A     Disabled
2696    820     taskeng.exe     0xfa8003a4b7c0  5       81      0       False   2026-02-27 16:53:25.000000 UTC  N/A     Disabled
```

We can see that all the process running are default but the cmd.exe and powershell.exe look little suspicious

We can try to get the command shell usages using the command **“vol -f memory.raw windows.cmdline”**



We can see that we have a encode string used with powershell, we can use cyberchef to decode it

By decoding the string with base64 we got the first half the flag

Now using the hints, we can try to extract scheduled task and some important things like adobeupdater and artifacts

```
┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ strings memory.raw | grep schtasks
schtasks.exe
schtasks.exe /delete
schtasks.exe /create /tn "defender monitor"

┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ strings -el memory.raw | grep -i adobeupdater
\AdobeUpdater
AdobeUpdater
indows\System32\Tasks\AdobeUpdater
 /create /sc onlogon /tn "AdobeUpdater" /tr "cmd /c echo 4[redacted]S_" /ru SYSTEM
\AdobeUpdater
\AdobeUpdater
schtasks  /run /tn "AdobeUpdater"
run /tn "AdobeUpdater"
"AdobeUpdater" /tr "cmd /c echo 4[redacted]S_" /ru SYSTEM
ws\system32>o run the scheduled task "AdobeUpdater".
- schtasks  /run /tn "AdobeUpdater"
"AdobeUpdater" /tr "cmd /c echo 4[redacted]S_" /ru SYSTEM
 /run /tn "AdobeUpdater"
AdobeUpdater
```

We can see that we have retrieved the second part of the flag in the abodeupdater

Now we can try to look at the artifacts

```
┌──(volatile)─(sibhath㉿sibhath)-[~/Documents]
└─$ strings -el memory.raw | grep -i artifact
Artifact tag: R[redacted]D
Artifact tag: R[redacted]D
Artifact tag: R[redacted]D
```

We can see that we have the third part of the flag retrieved from the artifacts

Now we can try to take a look at the environment variables for some hint by using the command **“vol -f memory.raw windows.envars”**

```
1224    notepad.exe     0x301de0        USERDOMAIN      WIN-12QU7UDUGN0
1224    notepad.exe     0x301de0        USERNAME        Guy
1224    notepad.exe     0x301de0        USERPROFILE     C:\Users\Guy
1224    notepad.exe     0x301de0        windir  C:\Windows
1224    notepad.exe     0x301de0        windows_tracing_flags   3
1224    notepad.exe     0x301de0        windows_tracing_logfile C:\BVTBin\Tests\installpackage\csilogfile.log
1724    cmd.exe 0x4b1d90        ALLUSERSPROFILE C:\ProgramData
1724    cmd.exe 0x4b1d90        APPDATA C:\Users\Guy\AppData\Roaming
1724    cmd.exe 0x4b1d90        CLEANFLAG       S[redacted]Y
1724    cmd.exe 0x4b1d90        CommonProgramFiles      C:\Program Files\Common Files
1724    cmd.exe 0x4b1d90        CommonProgramFiles(x86) C:\Program Files (x86)\Common Files
1724    cmd.exe 0x4b1d90        CommonProgramW6432      C:\Program Files\Common Files
1724    cmd.exe 0x4b1d90        COMPUTERNAME    WIN-12QU7UDUGN0
1724    cmd.exe 0x4b1d90        ComSpec C:\Windows\system32\cmd.exe
1724    cmd.exe 0x4b1d90        FP_NO_HOST_CHECK        NO
1724    cmd.exe 0x4b1d90        HOMEDRIVE       C:
1724    cmd.exe 0x4b1d90        HOMEPATH        \Users\Guy
1724    cmd.exe 0x4b1d90        LOCALAPPDATA    C:\Users\Guy\AppData\Local
1724    cmd.exe 0x4b1d90        LOGONSERVER     \\WIN-12QU7UDUGN0
1724    cmd.exe 0x4b1d90        NUMBER_OF_PROCESSORS    1
1724    cmd.exe 0x4b1d90        OS      Windows_NT
1724    cmd.exe 0x4b1d90        Path    C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\
1724    cmd.exe 0x4b1d90        PATHEXT .COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
1724    cmd.exe 0x4b1d90        PROCESSOR_ARCHITECTURE  AMD64
1724    cmd.exe 0x4b1d90        PROCESSOR_IDENTIFIER    Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
1724    cmd.exe 0x4b1d90        PROCESSOR_LEVEL 6
1724    cmd.exe 0x4b1d90        PROCESSOR_REVISION      8c01
1724    cmd.exe 0x4b1d90        ProgramData     C:\ProgramData
1724    cmd.exe 0x4b1d90        ProgramFiles    C:\Program Files
```

We can see that we have retrieved the last part of the flag we can combine them all and submit the flag